Perfilamiento, limpieza y normalización con Python

Práctica guiada 02

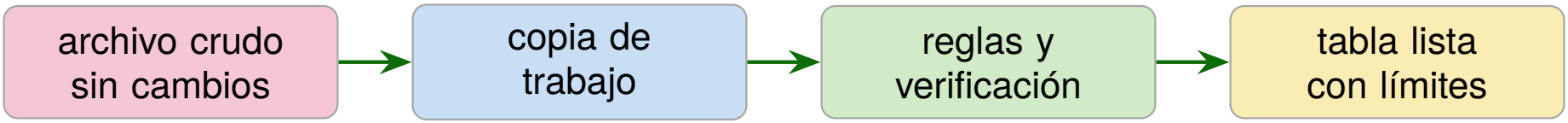
Manuel Soto Romero Araceli Liliana Reyes Cabello

Colegio de Ciencia y Tecnología, UACM

Semestre 2026-2

Datos crudos, copia de trabajo y línea base

La organización quiere comparar después los precios registrados por región y tipo de propiedad. Antes necesita una preparación que pueda auditarse.



La **línea base** registra filas, columnas, duplicados y variables críticas antes de modificar una representación.

Ejemplo: modificar una copia sin perder el origen

```
origen = pd.DataFrame({
    "registro": [1, 2],
    "estado": ["pendiente", "confirmado"]
})
trabajo = origen.copy(deep=True)
trabajo.loc[0, "estado"] = "revisar"
```

Resultado en el origen

El registro 1 continúa como `pendiente`.

Resultado en la copia

El registro 1 cambia a `revisar`.

Interpretación: podemos contrastar lo recibido con el resultado; la copia no reemplaza la bitácora de decisiones.

Reto 1

Crea una copia de trabajo, registra la forma inicial y los duplicados exactos, y define las columnas críticas con una condición de calidad para cada una.

Perfil de calidad orientado a la pregunta

Un perfil no es una colección indiscriminada de estadísticas: busca señales que puedan afectar la decisión.

- `dtype`: representación técnica actual.
- `isna().sum()`: cantidad de ausencias.
- `isna().mean()`: proporción de ausencias.
- `nunique()`: diversidad de valores observados.
- `duplicated().sum()`: repeticiones exactas.

Una señal puede ser **bloqueante**, **relevante** o quedar **pendiente de regla**; el código no elige esa interpretación.

Ejemplo: tres señales con efectos distintos

```
columnas = ["Price", "Date", "Type"]
perfil = pd.DataFrame({
  "tipo": datos[columnas].dtypes.astype(str),
  "faltantes": datos[columnas].isna().sum(),
  "porcentaje": datos[columnas].isna().mean() * 100,
  "distintos": datos[columnas].nunique()
})
```

- `Price` tiene ausencias: algunas filas no sirven para calcular una métrica de precio.
- `Date` aparece como texto: todavía debemos probar su conversión.
- `Type` no tiene faltantes: aún debemos contrastar sus códigos con el diccionario.

Interpretación: ausencia técnica, tipo técnico y validez semántica responden preguntas diferentes.

Reto 2

Construye el perfil de las seis columnas indicadas, cuenta duplicados exactos y clasifica cada hallazgo como bloqueante, relevante o pendiente de regla.

Normalización documentada

Normalizar lleva representaciones equivalentes a un formato común sin ocultar el valor recibido.

- `Date` se interpreta con el formato día/mes/año.
- `Type` se traduce sólo con tres equivalencias documentadas.
- `Method` se traduce sólo con los códigos proporcionados.

Creamos `Date_clean`, `Type_label` y `Method_label`; las columnas crudas permanecen disponibles.

Qué significa un fallo

`errors=çoerce"` y `map` hacen visible una entrada no interpretable como valor faltante. No la reparan ni autorizan una etiqueta inventada.

Ejemplo: conversión con evidencia visible

```
muestra["Date_clean"] = pd.to_datetime(
    muestra["Date"], format="%d/%m/%Y", errors="coerce"
)
muestra["Type_label"] = muestra["Type"].map(mapa_tipo)
muestra["Method_label"] = muestra["Method"].map(mapa_metodo)
```

`3/09/2016`, `h`, `S`
se convierten

`31/02/2017`, `x`, `ZZ`
quedan señalados

Interpretación: el resultado distingue equivalencias respaldadas de casos que todavía requieren documentación.

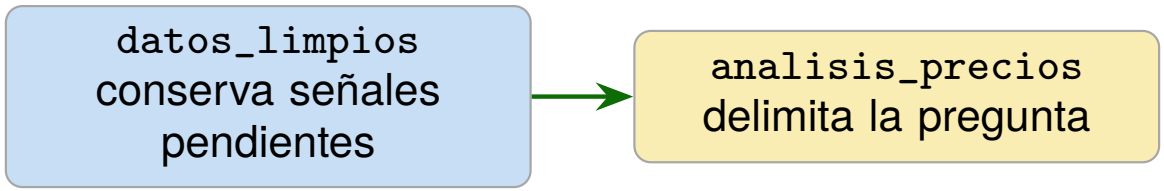
Reto 3

Crea columnas normalizadas para fecha, tipo y método; comprueba los fallos de conversión y los códigos no documentados sin sobrescribir las columnas de origen.

Una regla distinta para cada señal

Aplicaremos sólo reglas explícitas para la pregunta actual.

1. Retirar una repetición idéntica en las 21 columnas y conservar una fila.
2. No imputar `Price`: no existe una regla para reconstruirlo.
3. Conservar los demás faltantes en la tabla limpia.
4. Crear un subconjunto para precios con cuatro campos indispensables presentes.



Ejemplo: medir el impacto de las reglas

```
sin_duplicados = muestra.drop_duplicates().copy()
para_precio = sin_duplicados.dropna(
    subset=["precio", "region"]
).copy()
```

Recibidas

4 filas

Únicas

3 filas

Para precio

2 filas

Interpretación: una fila se retira por igualdad completa; otra permanece en la tabla limpia, pero queda fuera de la pregunta de precio. Un faltante no se convierte en cero.

Reto 4

Elimina sólo duplicados exactos en la tabla limpia, crea el subconjunto con las cuatro columnas indispensables y registra cuántas filas afecta cada regla sin imputar valores.

Validación antes del análisis

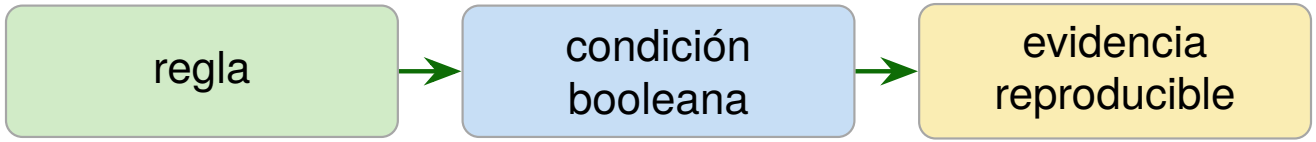
Una validación convierte reglas escritas en condiciones comprobables.

- El origen debe coincidir con la instantánea inicial.
- La tabla limpia no debe conservar duplicados exactos.
- Las fechas y códigos deben quedar convertidos o señalados.
- El subconjunto no debe tener faltantes en sus cuatro campos indispensables.

Una aserción verdadera confirma una condición concreta; no demuestra que los valores extremos sean correctos ni elimina los límites de cobertura.

Ejemplo: de una regla a una comprobación

```
comprobaciones = pd.Series({
  "sin_duplicados": sin_duplicados.duplicated().sum() == 0,
  "sin_precio_faltante": para_precio["precio"].notna().all(),
  "origen_conservado": len(muestra) == 4
})
assert comprobaciones.all()
```



Interpretación: la preparación puede aprobarse para una pregunta delimitada y conservar cautelas para lo que aún no se revisó.

Reto 5

Construye y ejecuta cinco comprobaciones, completa la bitácora y decide si el resultado está listo, listo con cautela o pendiente para el análisis posterior de precios.

Referencias

- McKinney, W. *Python for Data Analysis*. 3.ª ed., O'Reilly Media, 2022.
- BEDU. *Procesamiento de datos con Python*. Sesiones 4, 5 y 7, 2020–2021.
- Pino, T. *Melbourne Housing Market*. Kaggle; datos procedentes de Domain.com.au; licencia CC BY-NC-SA 4.0.
- Notas de clase 06, 07 y 08 de Inteligencia de Negocios.